

LLD Document

Low-Level Design

Module Implementation & Internal Architecture

Project	RAG-Based Customer Support Assistant
Technology	LangGraph + ChromaDB + Groq LLM + HuggingFace
Document Type	LLD Document
Date	April 2025

1. Module-Level Design

Document Processing Module ([pdf_loader.py](#))

Uses PyPDFLoader to read the PDF from disk. Returns a list of LangChain Document objects with page_content and metadata (source, page number). Handles file-not-found errors gracefully.

Chunking Module ([text_splitter.py](#))

Uses RecursiveCharacterTextSplitter with chunk_size=500, chunk_overlap=50. Overlap ensures context is not lost at chunk boundaries. Returns a list of smaller Document objects.

Embedding Module ([embeddings.py](#))

Uses HuggingFaceEmbeddings with model all-MiniLM-L6-v2. Converts text strings to 384-dimensional float vectors. Runs entirely locally — no API call required.

Vector Storage Module ([vector_store.py](#))

Uses Chroma.from_documents() to build the vector index. Persists to ./chroma_db directory so re-runs skip re-indexing. Supports similarity search with configurable k.

Retrieval Module ([retriever.py](#))

Wraps ChromaDB as a LangChain Retriever using db.as_retriever(search_kwargs={'k':3}). Returns top-3 most semantically similar chunks for any query.

Query Processing Module ([query_handler.py](#))

Formats the prompt template combining retrieved context + user query. Sends the formatted prompt to the Groq LLM. Parses and returns the text response.

Graph Execution Module ([graph.py](#))

Defines the LangGraph StateGraph with nodes: retrieve_node, generate_node, escalate_node. Compiles the graph and exposes app.invoke(state) as the main entry point.

HITL Module ([hitl.py](#))

Triggered when confidence < 0.5 or query contains escalation keywords. Displays escalation message, logs query for human review, and returns structured HITL response.

2. Data Structures

GraphState (TypedDict) — State object flowing between nodes:

```
class GraphState(TypedDict):  
    query: str # Original user question  
    context: str # Retrieved chunks joined as string  
    answer: str # Final answer to display
```

```
route: Literal['answer', 'escalate'] # Routing decision
confidence: float # 0.0 to 1.0 – retrieval quality score
```

Document Chunk Format:

```
Document(
    page_content = 'Return window is 7 days from delivery...',
    metadata = {'source': 'data/knowledge_base.pdf', 'page': 2}
)
```

Query-Response Schema:

Field	Type	Description
query	str	Raw user input text
context	str	Concatenated retrieved chunk texts
answer	str	LLM output or HITL escalation message
confidence	float	1.0 = sufficient context, 0.3 = insufficient
route	str	'generate' or 'escalate'

3. LangGraph Workflow Design

Graph Structure:

Node / Edge	Type	Description
retrieve_node	Node	Entry point. Calls ChromaDB retriever, sets context and confidence in state
route_decision()	Conditional Edge	Reads state.confidence and query keywords; returns 'generate' or 'escalate'
generate_node	Node	Builds prompt from context+query, calls Groq LLM, sets state.answer
escalate_node	Node	Sets HITL escalation message in state.answer, logs query
START → retrieve	Edge	Fixed — retrieve is always the first node
retrieve → [generate escalate]	Conditional	Determined by route_decision() output
generate → END	Edge	Fixed — generation always leads to output
escalate → END	Edge	Fixed — escalation always leads to output

4. Conditional Routing Logic

The route_decision() function reads the current GraphState after retrieval and applies two rules:

Condition	Trigger Value	Route	Reason
Low context volume	len(context) < 200 chars	escalate	PDF has no relevant info for this query
Confidence score	confidence < 0.5	escalate	Retrieval quality is insufficient
Escalation keywords	'urgent', 'legal', 'lawsuit', 'refund'	escalate	Query requires human judgment
Normal query	confidence >= 0.5, no keywords	generate	Sufficient context available for LLM

5. HITL (Human-in-the-Loop) Design

- Trigger:** Confidence < 0.5 OR query contains escalation keywords (urgent, legal, lawsuit)
- Action:** `escalate_node()` sets a structured message in `state.answer` with warning icon
- Output:** Displays: query recorded, human agent will follow up, original question echoed back
- Integration:** In production: escalation would write to a ticketing system (Jira/Zendesk) or send email
- Post-escalation:** Human reviews the query, provides answer, which can be fed back as new knowledge

6. Interface Design

- Input Format:**
Plain text string entered at CLI prompt. No JSON wrapper needed. Max recommended length: 500 characters.
- Output Format:**
Plain text answer printed to stdout. HITL escalation includes structured message with warning emoji, escalation notice, and echoed query.

7. Error Handling

Error Scenario	Handling Strategy
PDF file not found	<code>FileNotFoundException</code> caught; print helpful message; exit gracefully
No relevant chunks found	context will be empty; confidence set to 0.3; auto-escalates to HITL
LLM API failure (Groq)	try/except around <code>llm.invoke()</code> ; fallback message displayed; escalate
Empty user query	Stripped input checked; if empty, prompt again without processing
ChromaDB corruption	Delete <code>./chroma_db</code> directory; re-run script to rebuild index